DEPI
PROJECT DOCUMENTATION PACK

MediCare – Clinic Management & Appointment System

Planning • Requirements • Analysis & Design • Testing • Delivery

Prepared from the supplied MediCare technical documentation and DEPI documentation guidelines.

Version 1.0 | 20 October 2026

How to use this pack

This PDF is the concise print-ready companion to the editable DOCX. The DOCX contains fuller descriptions, Mermaid diagram sources, use-case descriptions, test templates and the complete delivery checklist.

- Source of truth: supplied MediCare technical documentation for scope, proposed architecture, requirements, database, tests and deployment design.
- DEPI checklist: supplied Project Documentation_Final.pdf.
- Do not claim deployment, test success, repository existence or supervisor approval without evidence.
- Replace [TEAM INPUT] placeholders and update status after real execution.

Submission status and priority

Deliverable	DEPI date	Priority/status
Planning & Management	16 Oct 2026	URGENT – overdue; draft prepared
Literature Review	16 Oct 2026	URGENT – feedback/rubric must be real
Requirements Gathering	16 Oct 2026	Drafted; team validation pending
System Analysis & Design	6 Nov 2026	Next milestone; design drafts included
Implementation / Source Code	30 Nov 2026	Actual code/repository not verified
Final Presentation, Testing & Reports	4 Dec 2026	Templates/slides prepared; evidence pending

1. Project proposal

MediCare is a proposed web-based clinic management platform connecting patients, doctors and administrators. It supports doctor discovery, appointment booking with server-side conflict prevention, doctor schedule management, digital visit records and prescriptions, notifications, and an administrative dashboard with analytics.

Business problem

- Phone/paper appointments can lead to double bookings, missed reminders and poor visibility of doctor availability.
- Patient history may be scattered across paper files or disconnected tools.
- Clinic administrators need consistent data to manage users, schedules and reports.

Objectives and targets

- Target zero double bookings in concurrency tests.
- Provide online doctor availability and appointment management.
- Protect medical information using role/resource-level authorization and private file storage.
- Validate performance, reliability, security and usability through recorded tests; targets are not results until measured.

Scope and limitations

MVP: authentication/roles, doctor directory and schedule, appointment management, patient/doctor dashboards, records/documents/prescriptions, notifications, admin management, core analytics, tests and deployment documentation. The technical source lists payments/invoicing, insurance, waiting list, multi-clinic, FHIR export, staff MFA, malware scanning, advanced analytics and mobile client as future enhancements. The prototype is not claimed to be a certified medical device or legally certified e-prescribing platform.

2. Planning & management

Phase	Target	Deliverables
Planning & Requirements	Recovery sprint – immediate	Proposal, scope, stakeholders, user stories and requirements
System Analysis & Design	6 Nov 2026	Diagrams, schema, wireframes, API/deployment design

Phase	Target	Deliverables
Core implementation	7–23 Nov (proposed)	Auth, roles, directory, schedule, booking, dashboards and medical modules
Testing/stabilization	24 Nov–2 Dec (proposed)	Security, regression, bug fixes, user manual and README
Final delivery	4 Dec 2026	Presentation, infographic, final report and evidence audit

Team roles (assign actual names)

- Project coordinator: milestones, risks, meeting notes and supervisor communication.
- Business analyst/documentation: proposal, requirements and traceability.
- Backend/database lead: services, EF Core, SQL schema, migrations and authorization.
- Frontend/UI lead: responsive views, forms, calendar and dashboards.
- QA/DevOps lead: test plan, bug log, CI/build checks and deployment evidence.
- Presentation/user guide owner: slides, user manual, demo and infographic.

Risk register

Risk	Impact	Mitigation
Late start / overdue planning items	High	Agree on a recovery plan today; freeze MVP; assign owners and short daily check-ins.
Scope too large for deadline	High	Prioritize Must requirements; move payments, insurance, multi-clinic and mobile client to future work.
Double booking under concurrent requests	High	Use database constraints/transaction strategy and RowVersion; add concurrent booking tests.
Sensitive medical data exposure	High	Enforce resource-level authorization, private file storage, least privilege, audit logs and security tests.
External email/SMS provider unavailable	Medium	Use outbox/retry strategy and fake providers in tests; do not lose the business transaction.
Azure/configuration deployment delay	High	Deploy a minimal skeleton early; validate environment variables, secrets, DB connectivity and health checks
Unclear ownership or merge conflicts	Medium	Use feature branches, pull requests, code review and meaningful commits.
Unverified assumptions in documentation	Medium	Label assumptions, verify against code and supervisor feedback before final submission.

KPIs

- Booking integrity: zero double bookings in concurrency tests.
- Performance: proposed p95 under 2 seconds for directory, slots and dashboards with 50 concurrent users.
- Security: no open critical/high authorization or private-file defects before demo.
- Reliability: notification retries up to 5 times with backoff in the proposed design.
- Usability: booking in at most 4 interactions from directory (target).
- Traceability: each Must requirement linked to code and passing test before demo.

3. Requirements gathering

Stakeholders: Patients, Doctors, Clinic Administrators, clinic operations/supervisor and system operators/developers.

User story examples

- Patient: search doctors by specialization; book/cancel/reschedule a visit; view own records and prescriptions.
- Doctor: manage working hours/vacations; confirm/reject requests; document visits and create prescriptions.
- Admin: approve/suspend doctors; manage patients, specializations, settings and reports.
- System: queue and retry reminders so provider failures do not lose notification work.

Functional requirements

- FR-AUTH: Register/login, identity management, roles and access control; admin accounts provisioned.
- FR-DOC: Search/filter doctor directory and view doctor profile/specialization.
- FR-SCHED: Doctors manage working hours and vacations; system generates valid available slots.

- FR-APT: Patients book/cancel/reschedule; doctors confirm/reject; server-side checks prevent conflicts.
- FR-MED: Authorized access to medical records and documents; secure upload/download handling.
- FR-PRES: Doctors create and manage prescriptions; patients can view/print their own prescriptions.
- FR-NOTIF: Queue in-app/email notifications and reminders; SMS only if configured and consented.
- FR-PAT: Patient dashboard, appointments, profile, records, prescriptions and documents.
- FR-ADMIN: Admin manages doctor approval, patients, specializations, appointments and settings.
- FR-RPT: Admin dashboard and core operational analytics/reports.
- FR-OPS: Documented deployment, health/smoke checks and backup/recovery approach.

Non-functional requirements

- Security & privacy: HTTPS/HSTS, Identity password hashing and lockout, antiforgery, least privilege, resource authorization, private file storage, no clinical content in logs.
- Performance: Target p95 under 2 seconds for directory, slots and dashboards with 50 concurrent users; paginated list queries.
- Reliability: Outbox records and retry with backoff for notification delivery; provider failure must not lose the business transaction.
- Maintainability: Layer rules, nullable reference types, async end-to-end and analyzers in CI.
- Usability & accessibility: Booking target of at most 4 interactions from directory; clear validation/conflict messages; semantic HTML, keyboard access and contrast.
- Responsive design: Usable at 360 px width; tables scroll or collapse on mobile.
- Data integrity: Foreign keys, no cascade delete on clinical history, RowVersion on Appointment, unique filtered index against double booking.
- Backup/recovery: Azure SQL point-in-time restore, blob soft-delete/versioning and documented restore drill (proposed target).

4. System analysis & design

Architecture

- MediCare.Web: MVC controllers, Razor views, ViewModels, SignalR hubs and composition root.
- MediCare.Services: business logic, DTOs, validation, notification providers and reminder worker.
- MediCare.Data: entities, EF Core, repositories, Unit of Work, migrations and seeding.
- MediCare.Tests: xUnit unit, integration and architecture tests.

Core entities

ApplicationUser/Identity, PatientProfile, DoctorProfile, Specialization, ClinicSetting, Appointment, MedicalRecord, MedicalDocument, Prescription, Notification, NotificationPreference and AuditLog. Verify exact attributes/cardinalities against final migrations.

Technology stack

- ASP.NET Core MVC
- Entity Framework Core
- ASP.NET Core Identity
- SQL Server / Azure SQL Database
- Azure App Service and Azure storage/services as described in the technical design
- SignalR for in-app real-time notifications
- SendGrid (email) and Twilio (SMS, if enabled)
- xUnit for automated tests
- Bootstrap, FullCalendar and Chart.js for UI components

Diagram checklist

- Use-case diagram and use-case descriptions.
- ERD and logical/physical schema with normalization notes.
- Context and Level 1 DFD.
- Appointment booking sequence diagram; booking activity diagram; appointment state diagram; conceptual class diagram.

- UI wireframes for patient, doctor and admin journeys.
- Component and deployment diagrams; API documentation based on actual routes.

5. Testing & quality assurance

The technical document defines a proposed testing strategy; this pack does not claim the tests have run. Record the actual date, build, result and evidence for every case.

Test ID	Scenario	Expected result	Status
TC-AUTH-01	Valid user logs in	User reaches permitted page	Not run
TC-SEC-01	Patient attempts to open another patient's record	Access denied; no data disclosed	Not run
TC-SEC-02	Unauthenticated user requests a medical document URL	File is not publicly accessible	Not run
TC-APT-01	Patient books a valid available slot	Appointment is persisted with expected state	Not run
TC-APT-04	Two patients attempt the same slot concurrently	At most one booking succeeds; other receives conflict	Not run
TC-APT-05	Patient reschedules to an available slot	Old/new slot states are consistent; rules enforced	Not run
TC-DOC-01	Doctor creates overlapping/invalid working hours	Validation rejects invalid schedule	Not run
TC-MED-01	Authorized doctor views related patient's record	Record available only to authorized role	Not run
TC-PRES-01	Doctor creates a prescription	Saved and visible to the correct patient	Not run
TC-NOT-01	Appointment confirmation queues notification	Outbox item is created with deduplication behavior	Not run
TC-NOT-03	Email/SMS provider fails	Business transaction persists; notification retries per policy	Not run
TC-ADM-01	Non-admin opens admin route	Access denied	Not run
TC-OPS-01	Run post-deploy smoke test	Home, login, health and core journey pass	Not run
TC-PERF-01	Load directory/slots/dashboard with target concurrency	p95 target evaluated and result recorded	Not run

6. Literature review / evaluation record

The DEPI guideline asks for lecturer feedback, suggested improvements and final grading criteria. These were not included in the supplied project design and must not be fabricated.

- Lecturer feedback: [Paste actual feedback, date and reviewer].
- Suggested improvements: [Record request, owner, due date and status].
- Final grading criteria: [Insert official rubric/marks; if unavailable, state “Not yet provided”.]
- If a traditional literature review is required, add credible sources on clinic appointment systems, privacy, scheduling concurrency and usability.

7. Implementation, README and repository

- Confirm the four-project solution and layer dependency rules.
- Use feature branches, pull requests, code review and meaningful commits.
- Never commit secrets; use local secrets or environment variables.
- Validate migrations, seed data, tests and a clean-clone build.
- README must include exact SDK, prerequisites, configuration, migration commands, run/test commands, integrations and limitations.
- GitHub URL: [TEAM INPUT]. Deployment URL: [TEAM INPUT, only if live and tested].

8. User manual starter

Patient: sign in → search doctor → choose available slot → confirm booking → manage appointments → view own records/prescriptions/documents.

Doctor: sign in → manage working hours/vacations → review calendar → confirm/reject requests → document visits and create prescriptions.

Admin: sign in → approve/manage doctors → manage patients and specializations → update settings → review reports.

Update exact buttons and screenshots after the UI is implemented.

9. Final presentation outline

- 1. Title: MediCare – Clinic Management & Appointment System; team, track and supervisor.
- 2. Problem: Manual/disconnected appointment handling; scheduling conflicts; scattered patient history.
- 3. Proposed solution: Role-based clinic platform and key workflows.
- 4. Users & features: Patient, Doctor, Admin, Reminder Scheduler.
- 5. Requirements & success measures: Key functional requirements, privacy and performance targets.
- 6. Architecture: Four-project N-Tier design and request flow.
- 7. Data model: Core entities and ERD.
- 8. Key workflow: Appointment booking and concurrency conflict handling.
- 9. UI walkthrough: Real screenshots only, once implementation exists.
- 10. Testing & quality: Test plan, actual results, defects fixed; distinguish planned from executed.
- 11. Deployment & limitations: Actual environment/URL if deployed; otherwise state planned status honestly.
- 12. Conclusion & future work: MVP value, known limitations and future enhancements.

10. Final delivery checklist

- Proposal, plan, roles, risk register and KPIs completed.
- Real lecturer feedback/rubric added.
- Requirements and traceability finalized.
- All required diagrams rendered and reviewed.
- Wireframes and actual UI screenshots added.
- Source builds; migrations and tests validated.
- Bug log, README, technical docs and user manual updated.
- GitHub link added; deployment URL only if live and tested.
- Presentation and infographic finalized; exact DEPI track confirmed.
- All documents uploaded to GitHub.

Important limitation

This pack is a documentation draft based on the supplied technical design. It cannot prove implementation, testing, instructor approval, repository availability or deployment. Update it with evidence from the real project before submitting.